2025

# Análisis y Diseño de Circuitos Integrados

Alumno: Blanco Suárez Nahuel Ernesto

Profesores: Ing. Scandaliaris Jorge

Ing. Miranda Bonomi Fernando

# Índice

Resumen ..... 2
1. Introducción ..... 2
1.1.1 Marco conceptual de un flujo de diseño digital ..... 2
1.1.1.1 Process Design Kit y Electronic Design Automation ..... 6
1.1.1.2 PDK, EDA y LibreLane ..... 8
1.1.1.3 IIC-OSIC-TOOLS ..... 8
1.1.2 Utilización de LibreLane, herramientas EDA y PDK en la asignatura (Distribución IIC-OSIC-TOOLS) ..... 8
1.1.2.1 Aplicación práctica ..... 8
1.1.2.2 Experiencia personal con el uso de herramientas y en línea de comando ..... 10
1.1.3 Otras herramientas utilizadas durante la asignatura ..... 11
1.1.3.1 Experiencia personal Visual Studio Code ..... 11
1.2 VHDL Objetivos ..... 11
1.3 LibreLane Objetivos ..... 11
2. Materiales y Métodos ..... 12
2.2 Front - end ..... 12
2.3 Back - end ..... 14
3. Resultados ..... 15
3.2 Front – end ..... 15
3.3 Back - end ..... 17
01-yosys-vhdlsynthesis ..... 17
07-openroad-staprepnr ..... 17
08-openroad-floorplan ..... 18
14-openroad-generatepdn ..... 19
17-openroad-globalplacementskipio ..... 19
18-openroad-ioplacement ..... 20
21-openroad-globalplacement ..... 20
26-openroad-detailedplacement ..... 21
27-openroad-cts y 28-openroad-stamidpnr-1 ..... 21
47-openroad-stapostpnr ..... 22
48-openroad-irdropreport ..... 23
49-magic-streamout y 50-klayout-streamout ..... 23
final ..... 24
Conclusión ..... 24
Referencias ..... 25

# Resumen

A lo largo de la historia, las necesidades humanas inicialmente se resolvían de manera manual. Con el avance tecnológico, estas tareas comenzaron a realizarse mediante dispositivos eléctricos, en los cuales la información era procesada principalmente de forma analógica. Posteriormente, el desarrollo de los circuitos digitales marcó un punto de inflexión, permitiendo el procesamiento de información en forma binaria y facilitando una mayor robustez, precisión y capacidad de integración. La evolución de esta tecnología dio lugar a los circuitos integrados, capaces de concentrar sistemas digitales completos en un pequeño chip de silicio. Con el paso del tiempo, el avance en los procesos de fabricación permitió reducir progresivamente el tamaño de los dispositivos, aumentando simultáneamente su capacidad de procesamiento y eficiencia. En la actualidad, los circuitos integrados cumplen un rol fundamental en múltiples áreas como la medicina, las telecomunicaciones, la robótica entre otras. Debido a su relevancia tecnológica e industrial, resulta fundamental estudiar el proceso de diseño de estos dispositivos. La creación de un circuito integrado digital se lleva a cabo mediante un flujo de diseño, entendido como el conjunto de etapas, herramientas y metodologías que permiten transformar una especificación inicial en un chip físicamente fabricable. Este proceso se basa en la comprensión integral de las especificaciones del sistema, los requisitos funcionales, las restricciones de consumo y los objetivos de rendimiento, especialmente en el contexto de los ASIC (Application-Specific Integrated Circuits). Existen diversos flujos de diseño, tanto comerciales como de código abierto. Dentro de estos últimos se destaca LibreLane, una infraestructura flexible que permite la construcción de flujos de implementación física para ASIC digitales utilizando herramientas EDA (Electronic Design Automation). Por lo tanto, en el presente proyecto se abordará el diseño de un controlador de semáforo para una intersección de dos calles implementando el flujo de diseño basado en LibreLane, y empleando el proceso tecnológico SkyWater SKY130 (sky130A). Con el objetivo de adquirir dominio en la utilización del flujo de diseño, desarrollar criterio técnico y lograr una comprensión de los resultados obtenidos, entendiendo la importancia del análisis y diseño de circuitos integrados en el ámbito tecnológico.

# 1. Introducción

### 1.1.1 Marco conceptual de un flujo de diseño digital

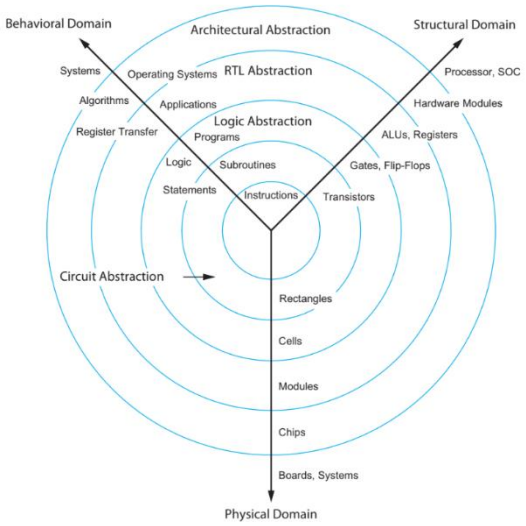
La manera en la que se aborda un diseño de un sistema, chip o circuito en particular puede tener un impacto profundo tanto en el esfuerzo invertido como en el resultado del diseño. Los diseñadores de Circuitos Integrados (IC) han desarrollado y adaptado estrategias de disciplinas aliadas, como la ingeniería de software, para formar un conjunto cohesivo de principios que aumenten la probabilidad de lograr diseños exitosos y a tiempo. Aunque los principios generales del diseño no han cambiado en décadas, los detalles de los estilos de diseño y las herramientas han evolucionado junto con los avances tecnológicos y los crecientes niveles de productividad. Un circuito integrado puede describirse en términos de tres dominios: el dominio conductual (behavioral), el dominio estructural, y el dominio físico.

- El dominio conductual se especifica lo que se busca en un sistema. En esta etapa no interesa la implementación física, sino su lógica funcional. Es la fase en la que interviene la programación, particularmente mediante la descripción de hardware en un lenguaje HDL.

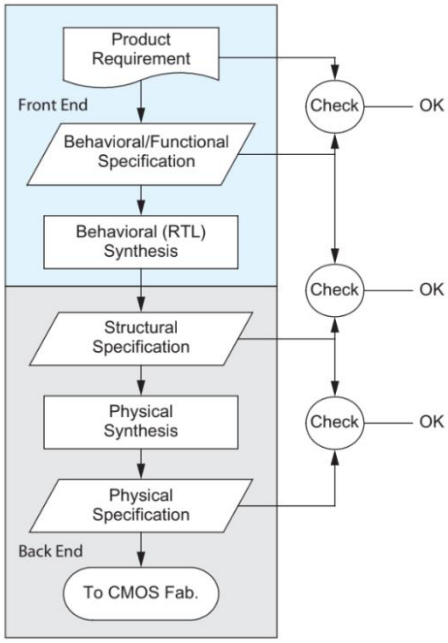
- El dominio estructural especifica la interconexión de los componentes necesarios para lograr el comportamiento deseado. En esta etapa, la descripción del comportamiento se traduce en una arquitectura compuesta por módulos, bloques o componentes que cumplen con los requisitos funcionales.

- El dominio físico, finalmente, especifica cómo se implementan y organizan físicamente los componentes para que puedan conectarse adecuadamente y cumplir el comportamiento requerido. Aquí se consideran las especificaciones tecnológicas, como el tamaño de los dispositivos, la longitud de canal y demás parámetros de fabricación.

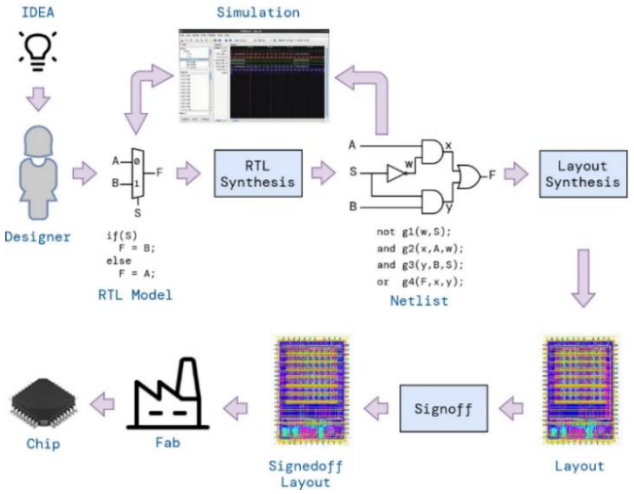
El flujo de diseño va desde el comportamiento hacia la estructura y, finalmente, a una implementación física a través de un conjunto de transformaciones manuales o automatizadas. Estos dominios pueden dividirse además jerárquicamente en diferentes niveles de abstracción de diseño. Clásicamente, para chips digitales suelen ser el nivel arquitectónico o funcional, lógico o de Transferencia de Registros (RTL), y nivel de circuito. La relación entre los dominios de descripción y los niveles de abstracción se muestra mediante el Diagrama en Y de Gajski-Kuhn



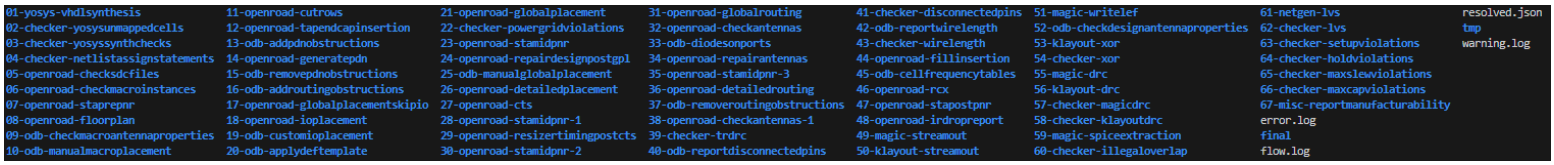

En sí un flujo de diseño es un conjunto de procedimientos que permite a los diseñadores progresar desde la especificación de un chip hasta la implementación final del mismo de manera libre de errores. En la figura se muestra un flujo de diseño general. El diseño comienza en el nivel conductual (behavioral), correspondiente a la Behavioral/Functional Specification, donde se definen los diagramas de estados, algoritmos, descripciones matemáticas y pseudocódigos. En esta etapa se especifica qué debe hacer el sistema, sin detallar aún su implementación física. Luego, el diseño procede al nivel estructural (puertas y registros), correspondiente a Behavioral (RTL) y Structural Specification, donde el HDL se traduce en una descripción RTL implementable. En este nivel el diseño queda expresado en términos de registros, lógica combinacional y transferencia de datos entre registros. Como resultado se obtiene un netlist, es decir, un archivo de texto que describe la conectividad de un circuito electrónico a nivel de compuertas o celdas estándar. Este paso se denomina síntesis behavioral o síntesis a nivel de transferencia de registros (RTL), porque los diseños capturados en un lenguaje de descripción de hardware (HDL) se transforman en una representación estructural compuesta por elementos de memoria y lógica. Posteriormente, la descripción lógica se transforma en una descripción física adecuada para la fabricación del chip, correspondiente a Physical Synthesis y Physical Specification. En esta etapa se pasa del netlist lógico a la implementación física mediante procesos como floorplanning, placement, clock tree synthesis y routing, hasta obtener el layout (representación geométrica del circuito integrado) y el archivo GDSII, que constituye la descripción final utilizada para la fabricación. Este paso se denomina síntesis física o layout generation. En la figura, el diseño se ha particionado en la etapa de front end (en el nivel conductual) y el back end (en los niveles estructural y físico). Esta partición es especialmente relevante en el diseño de Circuitos Integrados de Aplicación Específica (ASICs). En un flujo típico de ASIC, el diseño puede desarrollarse hasta el nivel HDL/RTL y luego transferirse a una empresa especializada que complete la transición hacia el chip físico. De este modo, la empresa de diseño original



no necesita invertir en las herramientas ni en el personal especializado requerido para las etapas de implementación física. Teóricamente, en un flujo de ASIC, el diseñador podría limitarse a desarrollar y verificar el HDL a nivel conductual/RTL, mientras que las etapas posteriores son realizadas por un servicio de diseño externo, siendo necesario verificar finalmente el cumplimiento de los requisitos de temporización (timing) durante el proceso de back-end.

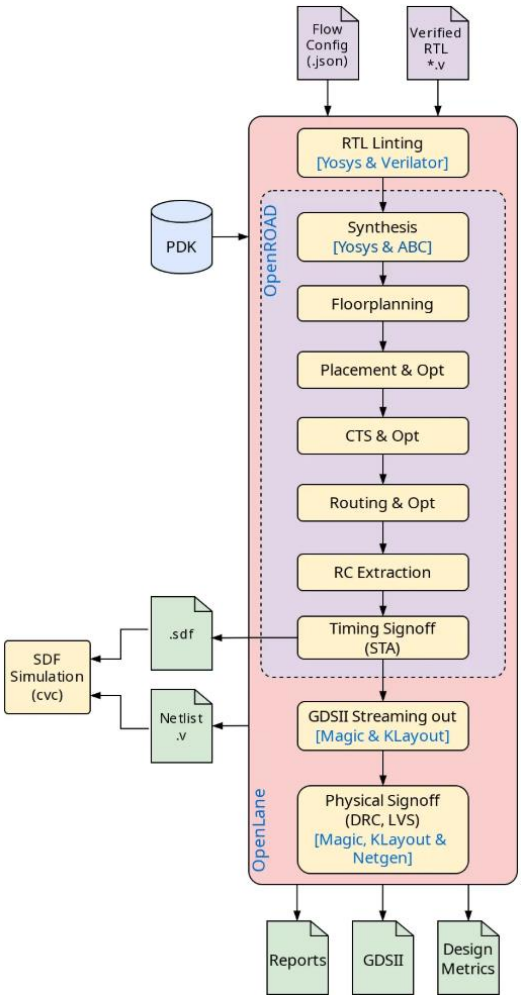



En este proyecto, entre los distintos flujos de diseño digital existentes, se utilizará LibreLane, una infraestructura robusta y versátil que permite la construcción de flujos de implementación de ASIC digitales basados en herramientas de código abierto, con posibilidad de integración de herramientas comerciales de EDA (Electronic Design Automation) permitiendo el diseño automatizado semi-custom. El análisis del flujo de diseño de LibreLane se fundamenta en su predecesor, OpenLane, plataforma de la




cual deriva, donde las diferencias no son en su flujo, sino más bien, en nuevas características, como la detección automatizada de errores sintácticos, la capacidad de crear flujos personalizados y una mayor precisión en la trazabilidad de fallos durante la ejecución. Aunque la arquitectura sigue un esquema general, esta se extiende a través de múltiples etapas automatizadas. Para el presente proyecto, se empleó el flujo VHDLClassic, el cual consta de 67 pasos predefinidos que pueden ser agrupados en distintos bloques operativos.

La primera etapa es la Front-end, donde al flujo ingresan el código VHDL ya verificado, junto con su archivo de configuración, en el cual se determinan algunos parámetros del chip a realizar. Antes de esto, se ingresa el PDK (Procces Design Kit) que utilizara el flujo. El primer bloque es RTL Linting, constituye el análisis el código RTL verificado, junto con el archivo de configuración del flujo, el objetivo es asegurar que el diseño esté limpio antes de avanzar hacia la síntesis, evitando propagar errores a etapas posteriores donde el costo de corrección sería mayor, mediante la detección de errores sintácticos y estructurales, la verificación de coherencia del código, la identificación temprana de construcciones no sintetizables y el chequeo de reglas básicas de diseño. Seguido del segundo bloque Synthesis, en esta etapa el diseño RTL se transforma en una netlist lógica compuesta por celdas estándar del PDK. A destacar el primer paso del flujo, *01-yosys-vhdlsynthesis*, el cual justamente se encarga de la síntesis y el mapeo tecnológico de los archivos VHDL utilizando Yosys, GHDL y ABC. Como resultado, el código VHDL se transforma en una netlist a nivel de compuertas (gate-level), mapeada a las celdas estándar disponibles en la librería tecnológica definida por el PDK, generando así una representación lógica dependiente de la tecnología seleccionada.




El siguiente paso, ya dentro del Back-end, es Floorplannig, que define la organización física inicial del chip. En este bloque se determinan las dimensiones del área del core, la ubicación de macros (si existen), la

definición de anillos de alimentación y la distribución preliminar de pines. En el flujo, el paso principal encargado de esta etapa es *08-openroad-floorplan*, el cual genera archivos DEF (Design Exchange Format) y ODB (OpenDB DataBase) con el plano inicial basado en la netlist generada por Yosys. El archivo DEF es una vista abstracta del diseño físico que describe la colocación de celdas instanciadas y macros, así como elementos en capas metálicas, incluidos pines y conexiones preliminares. Por su parte, el archivo ODB corresponde a la base de datos interna de OpenROAD (OpenDB), donde se almacena de manera estructurada toda la información del diseño físico, incluyendo instancias, geometrías, capas tecnológicas, reglas de diseño y conectividad, y constituye el formato de trabajo utilizado por OpenROAD en las etapas posteriores del flujo. En esta misma etapa se destaca el paso 14-openroad-generatepdn, encargado de construir la red de alimentación.

Continúa con Placement & Opt , donde el placement es la etapa del Back-end en la que se asigna una posición física (coordenadas X–Y) a cada celda estándar dentro del área del chip definida en el floorplan. Posteriormente, se realiza la optimización, cuyo objetivo es reducir la longitud de las interconexiones, minimizar la congestión, mejorar las métricas temporales y balancear área y rendimiento. En el flujo, se destaca el paso *21-openroad-globalplacement*, encargado de realizar la distribución global de las celdas, generando una colocación aproximada (no definitiva). Luego, el paso *26-openroad-detailedplacement* efectúa la colocación detallada y definitiva dentro de un archivo ODB.

CTS & Opt es el siguiente, el cual es la síntesis del árbol de reloj, y construye la red de distribución del clock en todo el chip, paso responsable *27-openroad-cts*.

Routing & Opt, etapa donde se realiza el ruteo físico de todas las interconexiones, tanto ruteo global, ruteo detallado y sus respectivas correcciones, uno de los pasos encargados es *31-openroad-globalrouting*, corresponde a la fase inicial de enrutamiento. A partir del archivo ODB con la colocación detallada ya legalizada, esta etapa asigna rutas a nivel de grano grueso (regiones o routing guides) para cada red, definiendo por qué áreas del chip deberá pasar cada interconexión. En esta fase no se generan aún los cables físicos definitivos, sino que se establecen las guías de enrutamiento que luego serán refinadas y materializadas durante el ruteo detallado.

RC Extraction, extrae parámetros parásitos, resistencias y capacitancias, 46-openroad-rcx.

Timing Signnoff (STA), en esta fase se ejecuta el Static Timing Analysis (STA), cuyo objetivo es verificar que el circuito cumpla los requisitos temporales bajo condiciones reales de operación. El STA (Análisis Temporal Estático) responde a la siguiente pregunta fundamental: ¿el chip puede funcionar a la frecuencia deseada sin que los datos lleguen demasiado tarde o demasiado temprano? A diferencia de una simulación dinámica, el STA analiza matemáticamente todos los caminos temporales del diseño sin necesidad de estímulos de prueba. Después del routing, ya se conocen los efectos físicos reales de las interconexiones, como la longitud de los cables, la resistencia, la capacitancia introducida por las capas metálicas y los parásitos asociados. Estos factores impactan directamente en los retardos de propagación de las señales, por lo que el STA evalúa el comportamiento temporal considerando esta información. El análisis verifica principalmente tres aspectos: el cumplimiento de tiempos de setup, asegurando que los datos lleguen antes del siguiente flanco de reloj; el cumplimiento de tiempos de hold, verificando que los datos no cambien demasiado pronto después del flanco de reloj; y la determinación de la frecuencia máxima operativa alcanzable según los retardos reales del circuito. Si los datos llegan tarde se produce un error de setup, y si cambian demasiado rápido se produce un error de hold. Como resultado de esta etapa se generan archivos como el .sdf (Standard Delay Format), utilizado para simulaciones temporales con retardos anotados, además de reportes detallados de análisis que indican márgenes, violaciones y desempeño temporal del diseño. El paso encargado de esto es el 47-openroad-stapostpnr el cual realiza tanto el análisis temporal y genera el archivo .sdf junto con los reportes de temporización.

SDF Simulation genera una simulación SDF, con otra herramienta compilando el netlist junto con el archivo .sdf y de esta manera generando un archivo CVC que permite ejecutar y visualizar la simulación con retardos reales.

GDSII Streaming out, en esta fase se produce el archivo GDSII, que contiene la descripción geométrica final del circuito listo para fabricación, utilizando herramientas como Magic y KLayout para el stream out del diseño físico. Los pasos encargados son *49-magic-streamout* y *50-klayout-streamout*

Finalmente, la etapa Physical Signoff (DRC, LVS) consiste en la verificación física completa del diseño antes de considerarlo apto para fabricación. Mediante el DRC (Design Rule Check) se validan las reglas tecnológicas del proceso de fabricación definidas por el PDK. En esta etapa se comprueba que el layout cumpla con las restricciones geométricas, tales como anchos mínimos de metal, separaciones entre capas, dimensiones de vías y ausencia de superposiciones ilegales. En el flujo, los pasos encargados de esta verificación son *55-magic.drc* y *56-klayout.drc*. Por otro lado, el LVS (Layout Versus Schematic), es decir, diseño versus esquemático, realiza la comprobación de equivalencia entre el layout físico y la netlist lógica del diseño. El responsable es el paso *61-netgen.lvs* es el encargado de verificar dicha equivalencia, comparando la netlist extraída del layout en formato .spice, con la netlist lógica del diseño, asegurando que ambas representaciones describan exactamente el mismo circuito.

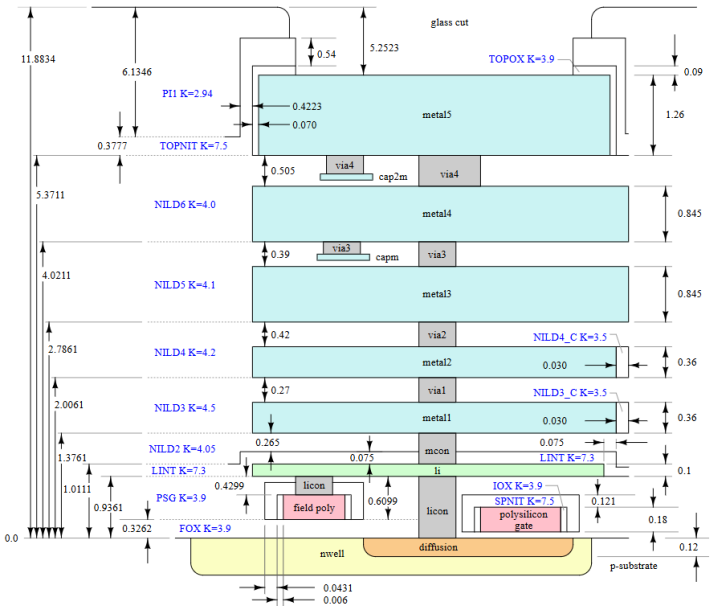
Al finalizar el flujo se obtienen reportes de métricas de diseño, la netlist final, el archivo SDF, el archivo GDSII y métricas de desempeño, como el área, el consumo de potencia, entre otros resultados. Los pasos del flujo no detallados corresponden, en su mayoría, a etapas de validación, corrección y generación de reportes intermedios necesarios para garantizar la integridad y calidad del diseño final.

#### 1.1.1.1 Process Design Kit y Electronic Design Automation

Anteriormente se describió el flujo de diseño empleado en LibreLane y, dentro de esa explicación, se mencionaron dos conceptos fundamentales para el desarrollo de circuitos integrados: Process Design Kit (PDK) y Electronic Design Automation (EDA). Para comprender con mayor profundidad el flujo completo, resulta necesario precisar el rol que cumple cada uno. En particular, el PDK constituye el vínculo técnico entre el diseño lógico y físico de un circuito integrado y su posterior fabricación en una fundición de semiconductores. El PDK puede definirse como el conjunto estructurado de archivos, bibliotecas y modelos que describen con precisión el comportamiento físico y eléctrico de un proceso tecnológico específico. En otras palabras, modela cómo se deben comportar los dispositivos reales como transistores, resistencias, capacitores e interconexiones metálicas, bajo las condiciones propias de una determinada tecnología de fabricación. Por esta razón, el PDK funciona como un “contrato” entre el diseñador y la fundición: garantizando que lo implementado mediante las herramientas EDA sea físicamente manufacturable y que, una vez fabricado, cumpla con las especificaciones de funcionamiento previstas en la etapa de simulación. El contenido de un PDK es integral y delimita el espacio de diseño disponible. Incluye modelos eléctricos de dispositivos en formato SPICE, que permiten simular el comportamiento de los transistores; reglas de diseño físico (Design Rule Check, DRC), que establecen restricciones geométricas mínimas tales como anchos de pista y espaciamientos para asegurar que las máscaras fotolitográficas puedan imprimirse sin defectos; reglas de verificación de correspondencia entre esquema y layout (Layout Versus Schematic, LVS), destinadas a comprobar que la implementación física respete exactamente la conectividad del diseño lógico; archivos de extracción parásita, que posibilitan estimar capacitancias e inductancias no ideales introducidas por las interconexiones; y bibliotecas de celdas estándar previamente caracterizadas en términos de área, retardo y consumo de potencia. Todos estos elementos son imprescindibles dentro de un flujo VLSI, ya que permiten cerrar el ciclo entre especificación, síntesis, implementación física y verificación. La importancia del PDK radica, en primer lugar, en la garantía de manufacturabilidad. Sin las reglas impuestas por la fundición, el diseñador podría generar geometrías imposibles de reproducir con la maquinaria real, lo que conduciría a dispositivos defectuosos. En segundo lugar, habilita la predicción de rendimiento eléctrico antes de incurrir en los costos de fabricación, reduciendo el riesgo económico asociado a iteraciones fallidas. Además, establece un marco estandarizado que permite la interoperabilidad entre diferentes herramientas EDA y equipos de trabajo. El cumplimiento estricto de sus reglas impacta directamente en el yield, es decir, en el porcentaje de circuitos funcionales obtenidos por oblea procesada. Los PDK son desarrollados y distribuidos exclusivamente por las fundiciones, tales como TSMC, Samsung

Electronics, Intel, SkyWater Technology o GlobalFoundries, ya que solo ellas poseen la información detallada sobre sus procesos litográficos, materiales y parámetros eléctricos reales. Debido a esta dependencia tecnológica, no es posible utilizar indistintamente un PDK en diferentes fábricas. Un diseño implementado para el proceso SKY130 de SkyWater, por ejemplo, no puede migrarse directamente a un proceso de 130 nm de otra empresa sin una adaptación profunda, dado que las reglas geométricas y los modelos de dispositivo difieren entre tecnologías. No todos los PDK presentan el mismo nivel de accesibilidad. En la industria de nodos avanzados, como 7 nm o inferiores, predominan los PDK propietarios, cuyo acceso está restringido mediante acuerdos de confidencialidad y cuya información constituye un activo estratégico de la empresa. En contraste, en los últimos años ha emergido una tendencia hacia PDK de código abierto, que publican sus reglas y modelos sin restricciones contractuales severas. Esta apertura ha posibilitado el desarrollo de herramientas libres y ha fortalecido la formación académica en microelectrónica. Un caso paradigmático es el PDK SKY130 promovido por SkyWater, así como el GF180MCU de GlobalFoundries. De este modo, el PDK no es un componente accesorio del proceso de diseño, sino el fundamento tecnológico que hace viable la transición desde la descripción en VHDL y su síntesis automatizada hasta la eventual fabricación física del circuito integrado.

En el presente proyecto se emplea el PDK sky130A, correspondiente a la familia SKY130 de código abierto. SKY130 es una tecnología híbrida madura de 180 nm - 130 nm desarrollada por Cypress Semiconductor, la cual se ha utilizado en múltiples piezas de producción. Actualmente, SKY130 está disponible como una tecnología de fundición a través de SkyWater Technology Foundry. Se trata de la 8.ª generación del nodo tecnológico SONOS (130nm). Especificaciones del Stack Tecnológico




- 5 niveles de metal ($p - penta$).
- Inductor o capacidad para inductores ($i$).
- Resistencia de polisilicio ($r$).
- Celda reducida SONOS ($s$).
- Soporte para fuente de alimentación regulada de 10V ($10R$).

El segundo concepto es EDA, un campo que abarca el desarrollo de software, hardware y servicios para automatizar el diseño de chips. LibreLane junto con todo el conjunto de herramientas que lo componen se consideran EDA; es decir, son todos aquellos recursos que ejecutan el flujo de diseño**.** Dentro del ecosistema de herramientas EDA utilizadas en LibreLane, cada componente cumple una función específica en alguna etapa del proceso de diseño y verificación de circuitos integrados.

**Yosys** es una herramienta de síntesis lógica que transforma la descripción en HDL en una representación estructural compuesta por compuertas lógicas y celdas estándar pertenecientes a la biblioteca del PDK.

**OpenROAD** es un conjunto integrado de herramientas que funcionan de manera coordinada dentro de un mismo entorno que realiza tareas como floorplanning, placement, clock tree synthesis, routing, análisis de temporización estática (STA) y optimización física, trabajando siempre bajo las restricciones tecnológicas del PDK.

**Odb (OpenDB)** es la base de datos de diseño empleada para almacenar y manipular la información estructural y física del circuito durante las distintas etapas del flujo.

**Magic** es una herramienta de edición y verificación de layout que se utiliza principalmente para la comprobación de reglas de diseño (DRC) y visualización geométrica del circuito integrado.

**KLayout** cumple funciones similares de visualización y verificación física, permitiendo inspeccionar layouts y ejecutar chequeos tecnológicos.

**Netgen** es una herramienta de verificación LVS (Layout Versus Schematic), cuya función es comparar el layout extraído con el esquema lógico para asegurar que ambas representaciones sean eléctricamente equivalentes.

**Checker** corresponde al conjunto de rutinas automáticas de verificación que evalúan condiciones específicas del diseño, como cumplimiento de reglas, consistencia de archivos o presencia de errores críticos.

**Pin Placer** es el módulo encargado de determinar la ubicación física de los pines de entrada y salida en el contorno del chip, respetando restricciones geométricas y eléctricas.

Yosys, OpenROAD, Magic, KLayout y Netgen constituyen, en conjunto, el núcleo de herramientas EDA que permiten transformar una descripción lógica en una implementación física verificable y manufacturable.

#### 1.1.1.2 PDK, EDA y LibreLane

En consecuencia, el PDK no ejecuta el diseño en sí mismo; las tareas operativas son realizadas por las herramientas EDA dentro del flujo de diseño. El PDK establece las reglas tecnológicas, modelos eléctricos y restricciones físicas propias del proceso de fabricación, mientras que las herramientas EDA implementan las distintas etapas del diseño respetando dichas restricciones. Por su parte, LibreLane actúa como el orquestador del proceso, coordinando y ejecutando cada herramienta en la secuencia adecuada para lograr una implementación correcta y verificable de un circuito integrado tipo ASIC.

#### 1.1.1.3 IIC-OSIC-TOOLS

Todo lo anteriormente descrito se encuentra integrado en una distribución denominada IIC-OSIC-TOOLS. Esta distribución consolida en un único entorno las herramientas EDA, el flujo de diseño y los PDK necesarios para el diseño de circuitos integrados, evitando la instalación y configuración individual de cada componente.

### *1.1.2 Utilización de LibreLane, herramientas EDA y PDK en la asignatura (Distribución IIC-OSIC-TOOLS)*

Al ser ICC-OSIC-TOOLS una distribución integral entre las herramientas EDA, los PDK y LibreLane, al instalar dicha distribución permitió disponer de un entorno unificado para el desarrollo completo de un diseño digital orientado a implementación ASIC, evitando la instalación individual de cada herramienta y la configuración manual de dependencias. La elección de esta distribución se considera adecuada principalmente por su carácter integrador y reproducible. Al disponer con dicha distribución se garantiza que todos los usuarios trabajen bajo el mismo entorno de versiones, bibliotecas y configuraciones, reduciendo problemas de compatibilidad, facilitando la portabilidad y la instalación en distintos equipos sin requerir conocimientos avanzados de configuración.

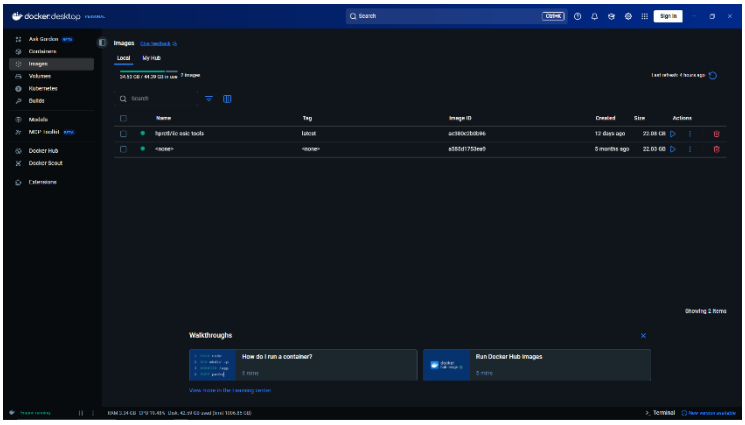
ICC-OSIC-TOOLS permite al usuario poder utilizar dichas herramientas de manera individual, sin necesidad de la ejecución del flujo. De esta manera, proporciona una mayor versatilidad y control sobre cada etapa del proceso

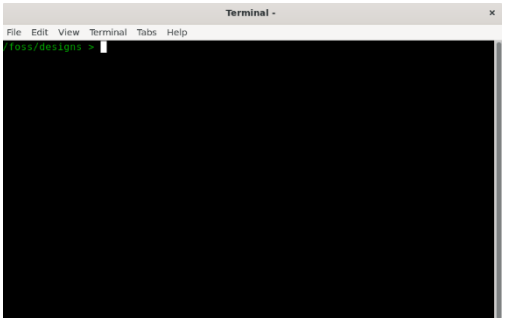
#### 1.1.2.1 Aplicación práctica

Dado que el flujo de diseño digital basado en LibreLane está orientado principalmente a entornos Linux, fue necesario incorporar una capa de virtualización que permitiera ejecutar las herramientas en un entorno compatible. Para resolver esta limitación, se instaló Docker Desktop, una plataforma de virtualización ligera basada en contenedores. Técnicamente, Docker no crea una máquina virtual completa en el sentido tradicional, sino que utiliza contenedores (instancia en ejecución de una imagen), los cuales son entornos aislados que comparten el kernel del sistema anfitrión, pero incluyen su propio sistema de archivos, bibliotecas, dependencias y herramientas.

En sistemas Windows, Docker Desktop utiliza internamente WSL2 (Windows Subsystem for Linux 2). En la práctica, esto implica que al ejecutar una imagen Linux desde Windows, se levanta un entorno Linux encapsulado dentro del sistema operativo anfitrión.

Una vez instalado Docker, la imagen (plantilla estática que contiene el sistema y las herramientas) de ICC-OSIC-TOOLS aparecerá en la sección Images. Al presionar el botón de ejecución, Docker instancia un contenedor activo a partir de dicha imagen. Al ejecutar la imagen hpretl/iic-osic-tools (latest), se abre una terminal que corresponde a un entorno Linux.




En las capturas puede observarse el prompt /foss/designs >, esto nos indica que el usuario se encuentra dentro del sistema de archivos del contenedor Linux. Mientras esta terminal permanezca abierta, el contenedor se mantiene en ejecución. Si la terminal se cierra, el contenedor se detiene.

Para cada diseño se crea una carpeta específica dentro de /foss/designs luego se crea una subcarpeta llamada src/ el cual será el directorio que contendrá los archivos VHDL. Paralelamente se genera un archivo config.yml (archivo configuración del flujo LibreLane) donde se declaran parámetros fundamentales como, nombre del diseño, tipo de flujo, archivos fuentes, restricciones de reloj, entre otros, según el diseño requerido.

```
! config.yml
1   meta:
2     flow: VHDLClassic
3
4   DESIGN_NAME: TOP_LEVEL
5
6   GHDL_ARGUMENTS:
7           - --std=08
8
9   VHDL_FILES:
10          - dir::scr/timer.vhd
11          - dir::scr/control.vhd
12          - dir::scr/TOP_LEVEL.vhd
13
```

```
/foss/designs/Timer > sak-pdk
Usage: sak-pdk <pdk> [<stdcell-lib>]

Available PDKs:
gf180mcuD
ihp-sg13g2
sky130A
```

Para la selección del PDK que utilizara el flujo, se puede especificar de dos maneras, mediante el archivo de configuración o en la línea de comandos

Una vez establecido config.yml junto al pdk a utilizar, se ejecuta el flujo mediante librelane config.yml

```
Terminal -
File  Edit  View  Terminal  Tabs  Help
/foss/designs/Timer > librelane config.yml
[08:14:11] INFO     Starting a new run of the 'VHDLClassic' flow      flow.py:651
                    with the tag 'RUN_2026-02-28_08-14-11'.
[08:14:11] INFO     Starting…                                   sequential.py:294
─────────────────────────── Synthesis (VHDL) ───────────────────────────
[08:14:11] VERBOSE  Running 'Yosys.VHDLSynthesis' at                 step.py:1140
                    'runs/RUN_2026-02-28_08-14-11/01-yosys-vhdlsynt
                    hesis'…
[08:14:12] VERBOSE  Logging subprocess to                            step.py:1336
                    'runs/RUN_2026-02-28_08-14-11/01-yosys-vhdlsynt
                    hesis/yosys-vhdlsynthesis.log'…

 /-----------------------------------------------------------------------------\
 |  yosys -- Yosys Open SYnthesis Suite                                        |
 |  Copyright (C) 2012 - 2025  Claire Xenia Wolf <claire@yosyshq.com>          |
 |  Distributed under an ISC-like license, type "license" to see terms          |
 \-----------------------------------------------------------------------------/
 Yosys 0.57 (git sha1 3aca86049, g++ 13.3.0-6ubuntu2~24.04 -fPIC -O3)

1. Executing Liberty frontend:
/foss/pdks/sky130A/libs.ref/sky130_fd_sc_hd/lib/sky130_fd_sc_hd__tt_025C_1v80.li
b
Imported 428 cell types from liberty file.
[INFO] Using SDC file
'/foss/designs/Timer/runs/RUN_2026-02-28_08-14-11/01-yosys-vhdlsynthesis/synthesis.abc.sdc' for ABC…

2. Executing GHDL.
Importing module TOP_LEVEL.
Importing module timer_8.
Importing module control_8.

3. Executing HIERARCHY pass (managing design hierarchy).

3.1. Analyzing design hierarchy..
Top module:  \TOP_LEVEL
Used module:     \control_8
Used module:     \timer_8

3.2. Analyzing design hierarchy..
Top module:  \TOP_LEVEL
Used module:     \control_8
```

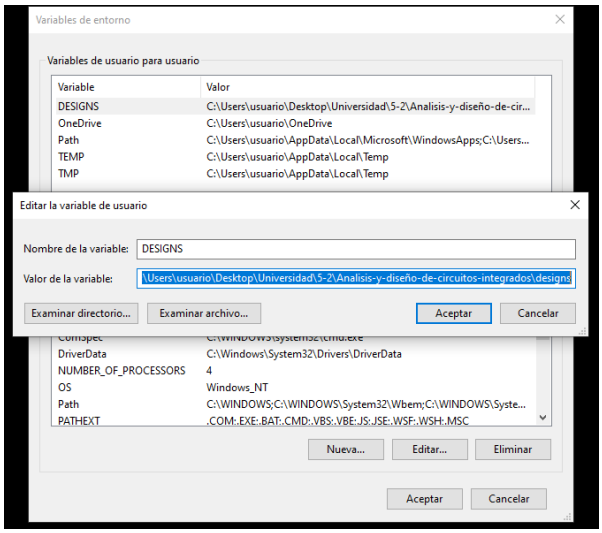
A partir de este momento, el sistema inicia automáticamente la secuencia completa de implementación física. Es importante remarcar que el usuario no interactúa manualmente con las herramientas individuales. Todas las herramientas EDA (Yosys, OpenROAD, Magic, etc.) están integradas dentro del flujo, y LibreLane orquesta su ejecución de forma secuencial y coherente. Si no se presentan errores de configuración, violaciones graves de temporización o conflictos de reglas de diseño, el flujo finaliza correctamente generando el layout final en formato GDSII.

#### 1.1.2.2 Experiencia personal con el uso de herramientas y en línea de comando

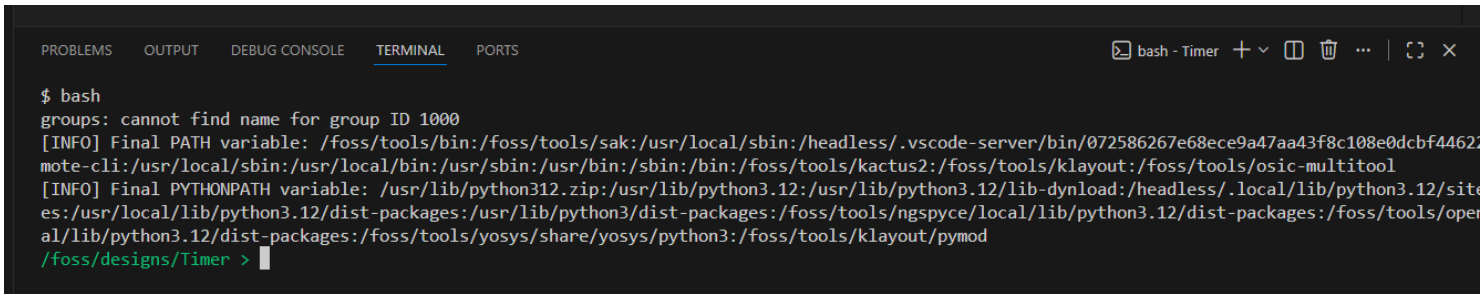
Desde un punto de vista personal, la experiencia fue positiva utilizando el entorno. Una de las principales ventajas fue no tener la necesidad de instalar individualmente cada herramienta, gracias a que la distribución IIC-OSIC-TOOLS integra todas las dependencias necesarias dentro de un mismo entorno. Asimismo, la ejecución del flujo resultó sencilla. Una vez que el código VHDL y el archivo config.yml estaban correctamente definidos, bastaba con ejecutar el flujo para que LibreLane automatizara cada una de las etapas del proceso. Esto eliminó la necesidad de realizar procedimientos manuales adicionales. En consecuencia, el flujo redujo sustancialmente la complejidad operativa, limitando la intervención a principalmente el análisis de resultados y a la eventual modificación del archivo de configuración cuando el diseño lo requería.

Como estudiante de Ingeniería Electrónica, mi formación ha tenido relativamente poca programación comparada con otras carreras. Mi experiencia previa provenía principalmente de materias de Ingeniería Biomédica y algunas asignaturas electivas previas. Específicamente con la línea de comandos se limitaba a un uso muy básico, principalmente relacionado con configuraciones de red. No tenía experiencia sostenida en entornos de línea de comando a diferencia de esta asignatura, que prácticamente toda la interacción se realizó desde una terminal.

### *1.1.3 Otras herramientas utilizadas durante la asignatura*




Se utilizó Visual Studio Code como entorno de desarrollo, para trabajar de forma más cómoda, tanto con los archivos VHDL como el flujo de diseño. Para ello se conecto Visual Studio Code directamente con Docker mediante la extensión Dev Containers. Y para una mayor flexibilidad se modificó la ubicación del directorio de trabajo. Por defecto este se encuentra dentro de la unidad del sistema (C:) en Windows. Para especificar una ruta alternativa, se configura la variable de entorno, y por medio de una terminal se ejecuta \start_x.bat para instalar e iniciar por primera vez el contenedor desde la nueva ubicación. Una vez conectado al contenedor desde Visual Studio Code y abierta la terminal integrada, al ejecutar el comando bash, se accede a una sesión interactiva del entorno Linux contenido. En este punto, el directorio de trabajo se posiciona automáticamente en /foss/designs. Esta ruta corresponde al directorio definido mediante la variable de entorno DESIGNS en Windows y montado como volumen dentro del contenedor. Por lo tanto, cualquier proyecto creado allí se almacena físicamente en el sistema anfitrión.




#### 1.1.3.1 Experiencia personal Visual Studio Code

En contraste con el trabajo en una terminal, las asignaturas previas que incluían programación utilizaban como entorno de desarrollo Visual Studio Code, por lo que ya existía una familiarización previa con dicha herramienta.

## *1.2 VHDL Objetivos*

Diseñar un controlador de semáforo para una intersección de dos calles. Se utilizará una máquina de estado finito jerárquica compuesta de una MEF de control para la secuencia de luces de los semáforos y un temporizador, MEF esclava, para medir el tiempo de duración de cada fase de la secuencia.

## *1.3 LibreLane Objetivos*

Implementación del flujo de diseño mediante el PDK sky130A

# 2. Materiales y Métodos

## 2.2 Front - end

Para la descripción de hardware de un controlador de semáforos para una intersección de dos calles, se utilizó el lenguaje VHDL 2008, desarrollando dos entidades funcionales timer.vhd y control.vhd, integradas mediante una entidad de nivel superior TOP_LEVEL.vhd. timer.vhd implementa un contador descendente que funciona como una entidad esclava del archivo contador.vhd. Este contador recibe un valor de recarga (reload) y, cuando está habilitado (enable), decrementa su cuenta en cada flanco ascendente del reloj (clk). Además, dispone de una señal de reinicio (rst) que fuerza el contador a cero.

```vhdl
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;


entity timer is
    generic( N : natural := 8);
    port (

        reload                  : in unsigned(N-1 downto 0);
        enable, rst, clk        : in std_logic;
        t, z                    : out std_logic
    );
end entity;

architecture timerLogic of timer is

    signal cuenta, cuenta_sig  : unsigned(N-1 downto 0);

    begin

        process (clk)

            begin

                if rising_edge(clk) then
                    cuenta <= cuenta_sig;
                end if;
        end process;

        cuenta_sig <= (others => '0')   when rst = '1'else
                      cuenta            when enable = '0' else
                      cuenta - 1        when not z else reload;

        t <= cuenta ?= 1;
        z <= cuenta ?= 0;


end architecture;
```

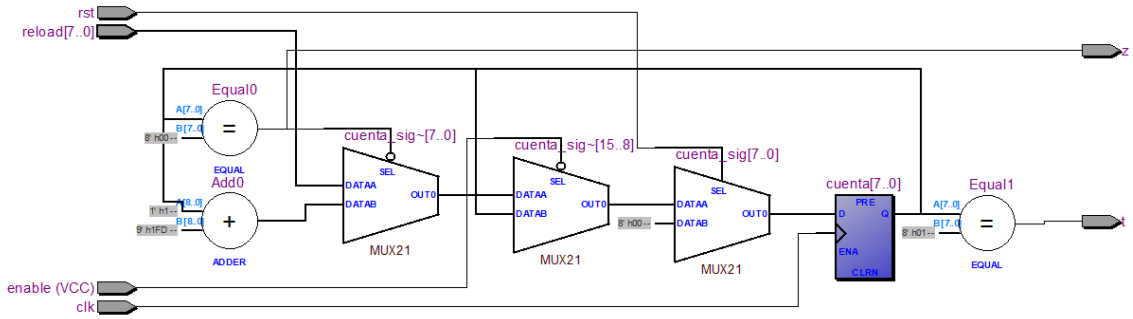
La primera parte del código correspondería a un flip-flop tipo D, sensible al flanco ascendente de reloj

```vhdl
process (clk)

    begin

        if rising_edge(clk) then
            cuenta <= cuenta_sig;
        end if;
end process;
```

La segunda parte del código describiría la lógica combinacional. Esta lógica puede interpretarse como una estructura de multiplexores.

```vhdl
cuenta_sig <= (others => '0')   when rst = '1'else
              cuenta            when enable = '0' else
              cuenta - 1        when not z else reload;

t <= cuenta ?= 1;
z <= cuenta ?= 0;
```

Obteniendo como resultado final




La imagen fue generada en Quartus basada en la arquitectura de una FPGA. La implementación física final en el flujo de diseño puede diferir de esta representación.

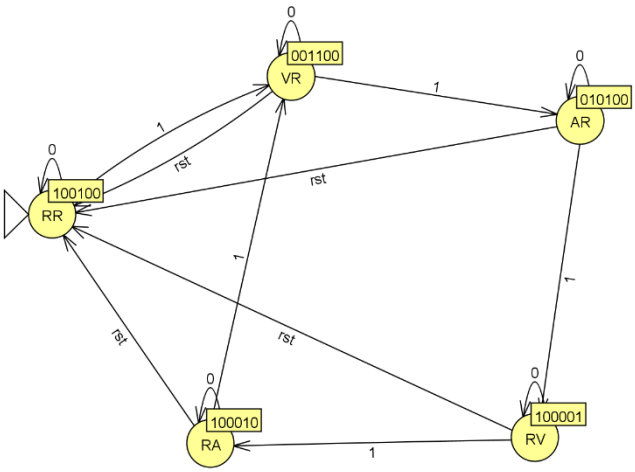
El siguiente código corresponde a control.vhd. Este archivo implementa una máquina de estados finitos (MEF) de tipo Moore, encargada de controlar los colores de los semáforos en la intersección.

La arquitectura define un tipo enumerado llamado estado, cuyos posibles valores son:

- RR (ambas calles en rojo)
- VR (verde en una calle, rojo en la otra)
- AR (amarillo en una calle, rojo en la otra)
- RV (rojo en la primera calle, verde en la segunda)
- RA (rojo en la primera calle, amarillo en la segunda)

Se utilizan dos señales, estado_act (almacena el estado actual) y estado_sig (representa el próximo estado). El proceso LES implementa la lógica de transición de estados y el bloque with select implementa la lógica de salida.

Dando como resultado el diagrama de estado




```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity control is
    generic( N : natural := 8);
    port(

        clk,rst,listo : in std_logic;
        luces : out std_logic_vector(5 downto 0);
        reload : out unsigned(N-1 downto 0)
    );
end entity;

architecture controlLogic of control is

        type estado is (RR,VR,AR,RV,RA);
        signal estado_sig, estado_act : estado;
        signal listo_i : std_logic;


        begin
        reload <= to_unsigned(2,N);

        ME : process(clk)
                begin
                    if rising_edge(clk)then
                        if rst = '1' then
                            estado_act <= RR;
                        else
                            estado_act <= estado_sig;
                        end if;
                    end if;
                end process;

        LES: process(all)
                begin
                    case estado_act is
                        when RR =>
                            if listo = '1' then estado_sig <= VR; else estado_sig <= RR;
                            end if;
                        when VR =>
                            if listo = '1' then estado_sig <= AR; else estado_sig <= VR;
                            end if;
                        when AR =>
                            if listo = '1' then estado_sig <= RV; else estado_sig <= AR;
                            end if;
                        when RV =>
                            if listo = '1' then estado_sig <= RA; else estado_sig <= RV;
                            end if;
                        when RA =>
                            if listo = '1' then estado_sig <= VR; else estado_sig <= RA;
                            end if;
                    end case;
                end process;

        LS: with estado_act select
                luces <= "100100" when RR,
                         "001100" when VR,
                         "010100" when AR,
                         "100001" when RV,
                         "100010" when RA,
                         "000000" when others;

    end architecture;
```

La primera parte del código genera un elemento secuencial síncrono, específicamente un registro dependiente del flanco ascendente del reloj.

```
ME : process(clk)
        begin
            if rising_edge(clk)then
                if rst = '1' then
                    estado_act <= RR;
                else
                    estado_act <= estado_sig;
                end if;
            end if;
        end process;
```

La segunda parte del diseño genera lógica combinacional

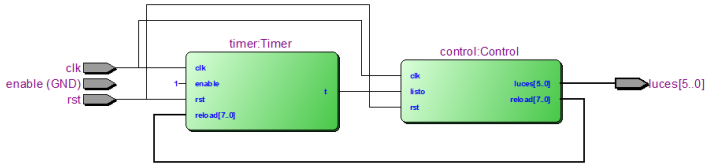
```
LES: process(all)
        begin
            case estado_act is
                when RR =>
                    if listo = '1' then estado_sig <= VR; else estado_sig <= RR;
                    end if;
                when VR =>
                    if listo = '1' then estado_sig <= AR; else estado_sig <= VR;
                    end if;
                when AR =>
                    if listo = '1' then estado_sig <= RV; else estado_sig <= AR;
                    end if;
                when RV =>
                    if listo = '1' then estado_sig <= RA; else estado_sig <= RV;
                    end if;
                when RA =>
                    if listo = '1' then estado_sig <= VR; else estado_sig <= RA;
                    end if;
            end case;
        end process;

LS: with estado_act select
        luces <= "100100" when RR,
                 "001100" when VR,
                 "010100" when AR,
                 "100001" when RV,
                 "100010" when RA,
                 "000000" when others;
```

Los elementos finales dependerán de la implementacion fisica mas adecuada determinada por el flujo de diseño

Finalmente, el ultimo codigo TOP_LEVEL.vhd, el cual es el encargado de intaciar las entidades anteriores y establecer la conexión funcional entre ambas. Ademas de declarar los pines de entrada y salida.

```vhdl
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity TOP_LEVEL is
    generic (N : natural := 8);
    port(

        clk,rst,enable : in std_logic;
        luces          : out std_logic_vector(5 downto 0)
    );
end TOP_LEVEL;

architecture TL of TOP_LEVEL is

    signal t,z : std_logic;
    signal reload_timer : unsigned(N-1 downto 0);

    begin

    Timer: entity work.timer
            generic map (N => N)
            port map (reload => reload_timer, enable => '1', rst => rst, z => z, t => t, clk => clk);

    Control: entity work.control
                generic map (N => N)
                port map (reload => reload_timer, rst => rst, listo => t, luces => luces, clk => clk);
end TL;
```

Obteniendo una representacion final




## 2.3 Back - end

Para hacer uso del flujo Librelane, se debe generar previamente un archivo config.yml, el cual contiene todos los parámetros necesarios para la construcción y síntesis del diseño.

```yaml
meta:
  flow: VHDLClassic

DESIGN_NAME: TOP_LEVEL

GHDL_ARGUMENTS: --std=08

VHDL_FILES:
            - dir::scr/timer.vhd
            - dir::scr/control.vhd
            - dir::scr/TOP_LEVEL.vhd

CLOCK_PORT: clk
CLOCK_PERIOD: 20

PDK_ROOT: /foss/pdks
PDK: sky130A
PDKPATH: /foss/pdks/sky130A
STD_CELL_LIBRARY: sky130_fd_sc_hd
```

- flow indica el flujo que se utilizará; en este caso, se emplea VHDLClassic, ya que el diseño está desarrollado en VHDL.
- DESIGN_NAME especifica el nombre de la entidad que será procesado por el flujo. Este debe corresponder al archivo de mayor jerarquía.
- GHDL_ARGUMENTS define los argumentos que se pasarán al compilador GHDL, como la versión del estándar VHDL utilizada. En este caso, --std=08 indica que se emplea el estándar VHDL-2008.
- CLOCK_PORT y CLOCK_PERIOD indica la conexión física del clock y el periodo del mismo
- VHDL_FILES contiene la lista de rutas donde se encuentran los archivos VHDL que conforman el diseño.

Para indicar a LibreLane que PDK utilizar, es necesario agregar las siguientes lineas en el archivo config.yml

```yaml
PDK_ROOT: /foss/pdks
PDK: sky130A
PDKPATH: /foss/pdks/sky130A
STD_CELL_LIBRARY: sky130_fd_sc_hd
```

```
/foss/designs/Timer > sak-pdk
Usage: sak-pdk <pdk> [<stdcell-lib>]

Available PDKs:
gf180mcuD
ihp-sg13g2
sky130A
```

PDKS disponibles en la distribución

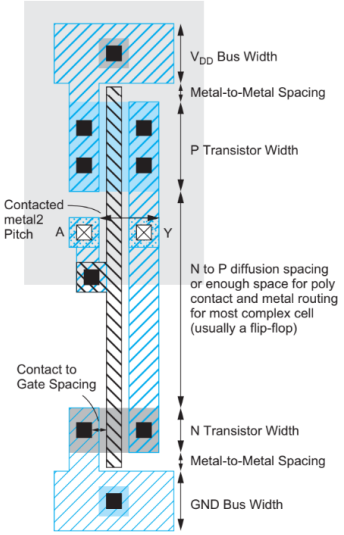
Donde

- PDK_ROOT: especifica la ruta raíz donde se encuentran instalados los PDKs en el sistema.
- PDK: indica el nombre del proceso tecnológico que se empleará. En este caso, se utiliza sky130A.
- PDKPATH: define la ruta completa hacia el PDK seleccionado.

- STD_CELL_LIBRARY: selecciona la biblioteca de celdas estándar que se usará durante la síntesis y el flujo físico; en este caso, sky130_fd_sc_hd, corresponde al proceso sky130, desarrollado por The SkyWater Foundry. Esta biblioteca pertenece al conjunto de celdas digitales estándar de alta densidad, optimizadas para reducir el área ocupada en el diseño.

| Architecture Comparison | Low Speed | High Density | High Density Low Leakage | High Voltage |
|---|---|---|---|---|
| TAP BAR | NO | NO | NO | YES |
| X-GRID | 0.480 | 0.460 | 0.460 | 0.480 |
| Y-GRID | 0.370 | 0.340 | 0.340 | 0.370 |
| CELL HEIGHT | 9 GRIDS | 8 GRIDS | 8 GRIDS | 11 GRIDS |
| CELL HEIGHT | 3.330 | 2.720 | 2.720 | 4.07 |
| NAND2 WIDTH | 3 GRIDS | 3 GRIDS | 4 GRIDS | 5 GRIDS |
| NAND2 WIDTH | 1.440 | 1.380 | 1.840 | 2.400 |
| NAND2 AREA | 4.7952 | 3.7536 | 5.0048 | 9.770 |
| WPMAX | 1.120 | 1.000 | 1.000 | 1.500 |
| WNMAX | 0.740 | 0.650 | 0.650 | 0.75 |

Comparación física y tecnológicas entre las distintas librerías estándar




- TAP BAR: Indica si la librería incluye barra de contacto a pozo/sustrato para evitar latch-up (conexión de well/substrate a VDD/GND)
- X-GRID: Paso mínimo horizontal
- Y-GRID: Paso mínimo vertical
- CELL HEIGHT: Altura total de la celda estándar expresada en unidades de grilla
- CELL HEIGHT: Altura física real de la celda entre VDD y GND
- NAND2 WIDTH: Ancho de una celda NAND2
- NAND2 WIDTH: Ancho físico real de la NAND2
- NAND2 AREA: Área total ocupada por la celda NAND2
- WPMAX: Ancho máximo permitido para el transistor PMOS dentro de la celda
- WNMAX: Ancho máximo permitido para el transistor NMOS dentro de la celda

Dado que el enunciado no impone restricciones de frecuencia, consumo o área, la selección de la biblioteca de celdas estándar se realizó considerando criterios de simplicidad, compatibilidad y uso académico. Por ende, se eligió la librería sky130_fd_sc_hd, por ser la opción por defecto en LibreLane. Dado que el sistema no requiere altas frecuencias de operación ni características especiales de bajo consumo o alto voltaje, no resulta necesario utilizar variantes como sky130_fd_sc_hs, sky130_fd_sc_lp o sky130_fd_sc_hvl.

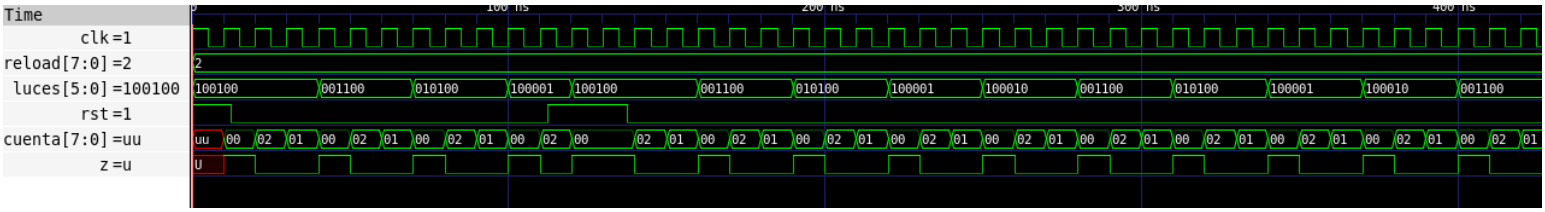
# 3. Resultados

## 3.2 Front – end

```
library ieee;
use ieee.std_logic_1164.all;
use std.env.all;

entity TOP_LEVEL_tb is
end entity;

architecture tb_TL of TOP_LEVEL_tb is

signal tbclk : std_logic := '1';
constant clk_half_period : time := 5 ns;

constant N : natural := 8;
signal rst, listo, enable : std_logic;
signal luces : std_logic_vector (5 downto 0);

    begin

    top_level: entity work.TOP_LEVEL
               generic map (N => N)
               port map (clk => tbclk, rst => rst, enable => enable, luces => luces);

    tbclk  <= not(tbclk) after clk_half_period;

    process
    begin

        rst  <= '1'; wait for  (clk_half_period * 5)/2;
        rst  <= '0';
        wait for 100 ns;
        rst  <= '1'; wait for  (clk_half_period * 10)/2;
        rst  <= '0'; wait for  (clk_half_period * 10)/2;
        wait for 300 ns;
        finish;

    end process;

end tb_TL;
```

Para realizar la comprobación del correcto funcionamiento de las entidades, se escribió el archivo TOP_LEVEL_tb.vhd, el cual corresponde al testbench (banco de pruebas o simulación) del sistema implementado. Este testbench

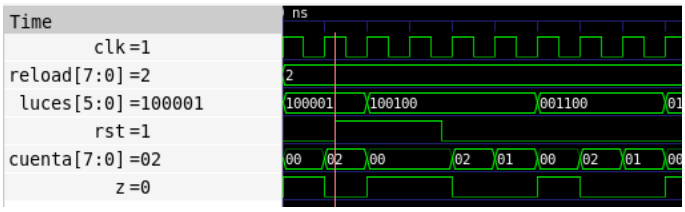
consiste en instanciar la entidad de nivel superior (TOP_LEVEL) y generar las señales de estímulo necesarias, con el fin de verificar mediante simulación el comportamiento funcional del diseño.




La máquina de estados finitos implementa la siguiente codificación

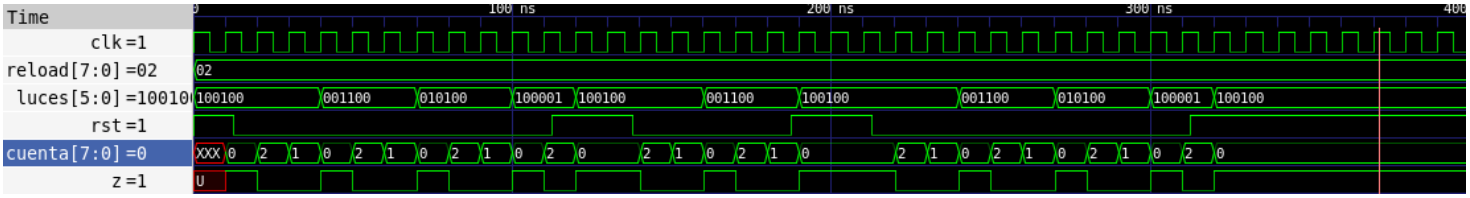
| Estado | luces [5:0] | Descripción |
|---|---|---|
| **RR** | 100100 | Rojo – Rojo |
| **VR** | 001100 | Verde – Rojo |
| **AR** | 010100 | Amarillo – Rojo |
| **RV** | 100001 | Rojo – Verde |
| **RA** | 100010 | Rojo – Amarillo |

La señal reload tiene un valor fijo de 2, esto implica que el cambio de estado ocurre cada 2 flancos ascendentes de reloj, el contador decrementa cada ciclo de clk, cuando el contador llega a cero, la salida del temporizador z se activa en 1. La señal z (listo en control) actúa como señal de habilitación para la máquina de estados, mientras z = 0, la MEF permanece en el estado actual, cuando z = 1, la MEF transita al estado siguiente. En la forma de onda se observa claramente que el patrón de luces cambia únicamente cuando z se activa, confirmando el correcto acoplamiento entre el temporizador y la MEF.

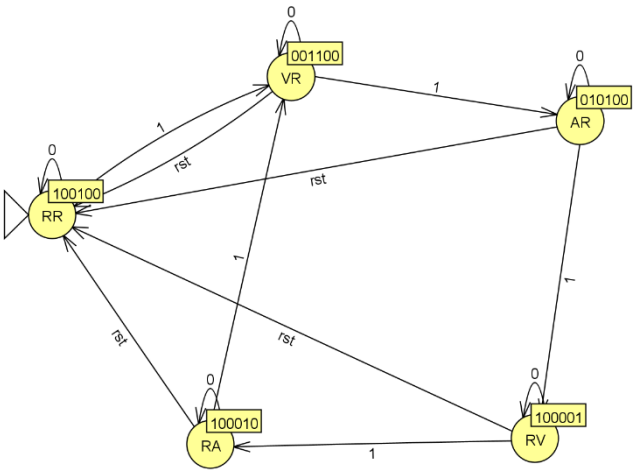



La tabla de estados según la simulación queda de la siguiente manera

| luces | Estado inicial | Estado siguiente | | |
|---|---|---|---|---|
| | | z = 0 | z = 1 | rst = 1 |
| **100100** | RR | RR | VR | RR |
| **001100** | VR | VR | AR | RR |
| **010100** | AR | AR | RV | RR |
| **100001** | RV | RV | RA | RR |
| **100010** | RA | RA | VR | RR |

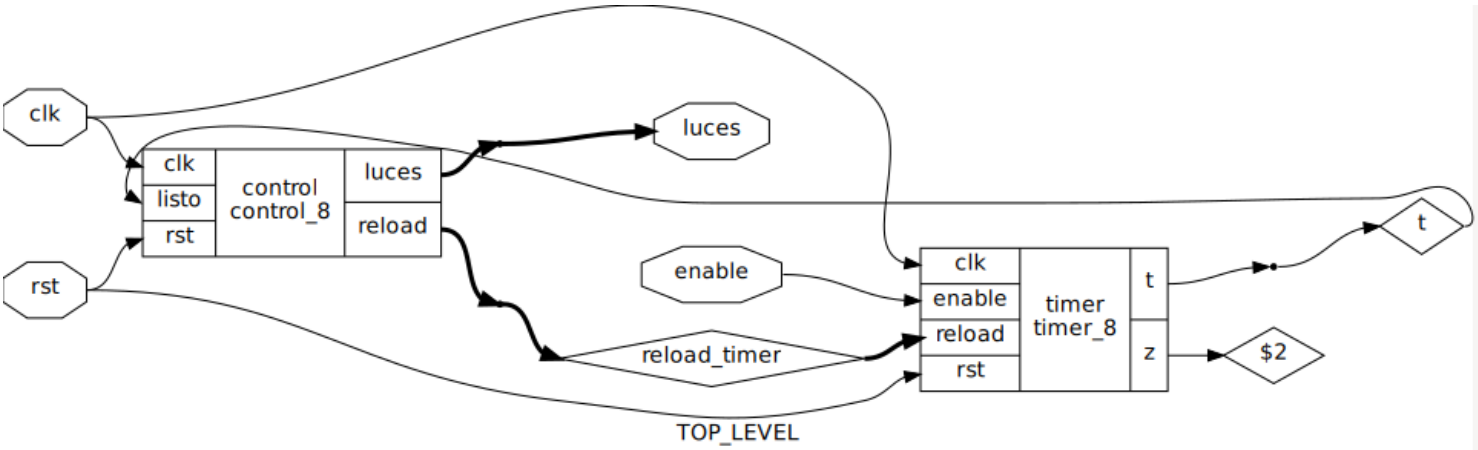



De esta manera, verificando el diagrama de estados



## 3.3 Back - end

### 01-yosys-vhdlsynthesis




| Tipo de componente | Nombre/Prefijo | Cantidad |
|---|---|---|
| **Buffers** | sky130_fd_sc_hd__buf_1 | 4 |
| **Inversores** | sky130_fd_sc_hd__inv_2 | 2 |
| **AND de 2 entradas (incluye negadas)** | sky130_fd_sc_hd__and2_2 / and2b_2 | 4 |
| **AND de 3 entradas** | sky130_fd_sc_hd__and3_2 | 2 |
| **AND de 4 entradas** | sky130_fd_sc_hd__and4_2 | 1 |
| **OR de 2 entradas** | sky130_fd_sc_hd__or2_2 | 1 |
| **OR de 3 entradas (incluye negadas)** | sky130_fd_sc_hd__or3_2 / or3b_2 | 4 |
| **OR de 4 entradas (incluye negadas)** | sky130_fd_sc_hd__or4_2 / or4b_2 | 2 |
| **NOR de 2 entradas** | sky130_fd_sc_hd__nor2_2 | 3 |
| **NOR de 4 entradas** | sky130_fd_sc_hd__nor4_2 | 1 |
| **NOR de 3/4 entradas con una negada** | sky130_fd_sc_hd__nor3b_2 / nor4b_2 | 2 |
| **NAND de 2 entradas (incluye negadas)** | sky130_fd_sc_hd__nand2_2 / nand2b_2 | 5 |
| **XOR de 2 entradas** | sky130_fd_sc_hd__xor2_2 | 1 |
| **Flip-Flops D** | sky130_fd_sc_hd__dfxtp_2 | 13 |
| **Combinacionales especiales (AIO, OAI, etc.)** | a21o, a22o, a31o, a21oi, o21ai, o31ai, o311a | 13 |

TOP_LEVEL.nl.v (Netlist Logico)

```
sky130_fd_sc_hd__nor4_2 _38_ (
  .A(\timer.cuenta[3] ),
  .B(\timer.cuenta[2] ),
  .C(\timer.cuenta[5] ),
  .D(\timer.cuenta[4] ),
  .Y(_15_)
);
```

Las celdas no cuentan con coordenadas físicas (x,y) y tampoco red física de alimentación

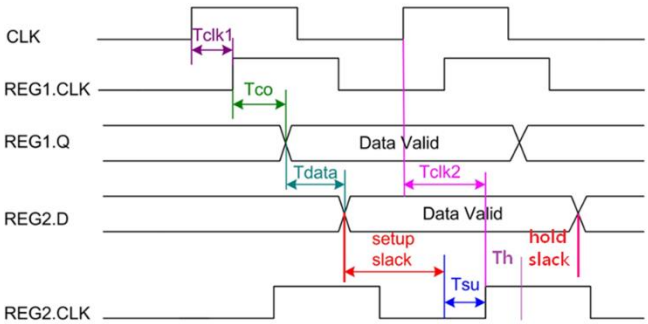
reports/latch.rpt: Sin latches generados

```
14. Executing PROC_DLATCH pass (convert process syncs to latches).
~
~
~
~
~
~
~
~
~
~
```

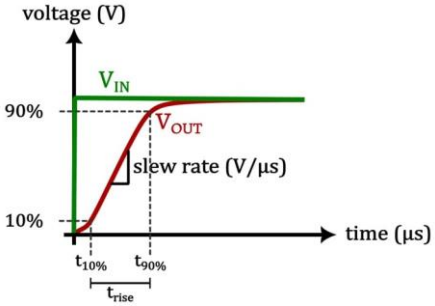
### 07-openroad-staprepnr

| Corner/Group | Hold Worst Slack | Reg to Reg Paths | Hold TNS | Hold Vio Count | of which reg to reg | Setup Worst Slack | Reg to Reg Paths | Setup TNS | Setup Vio Count | of which reg to reg | Max Cap Violatio… | Max Slew Violati… |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Overall | 0.0585 | 0.0585 | 0.0000 | 0 | 0 | 12.3611 | N/A | 0.0000 | 0 | 0 | 0 | 0 |
| nom_tt_025C_1v80 | 0.2286 | 0.2286 | 0.0000 | 0 | 0 | 14.2903 | N/A | 0.0000 | 0 | 0 | 0 | 0 |
| nom_ss_100C_1v60 | 0.6890 | 0.6890 | 0.0000 | 0 | 0 | 12.3611 | N/A | 0.0000 | 0 | 0 | 0 | 0 |
| nom_ff_n40C_1v95 | 0.0585 | 0.0585 | 0.0000 | 0 | 0 | 14.9281 | N/A | 0.0000 | 0 | 0 | 0 | 0 |

Slack - diferencia entre el tiempo disponible para que una señal llegue a su destino y el tiempo que realmente tarda




Data Valid es el período en el que el valor llegó al registro y se encuentra estable. Setup Slack es el tiempo sobrante desde que el valor ya está estable hasta que llega el flanco de reloj y el flip-flop realiza la captura. Hold Slack es el tiempo que el valor debe mantenerse estable después de la captura del reloj; si el valor cambia antes de que transcurra ese tiempo mínimo, el flip-flop no puede leerlo correctamente.

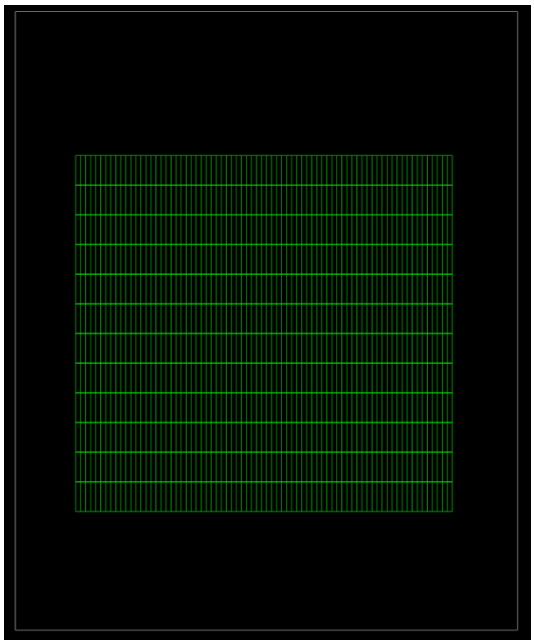



Los valores presentados en la tabla corresponden al análisis temporal del diseño bajo distintas condiciones de esquina (corners). Se observa que no existen violaciones de setup ni de hold, ya que los campos Setup TNS, Hold TNS y los contadores de violaciones se encuentran en cero. Además, el Worst Slack tanto para setup como para hold es positivo, en todos los casos, por ende, cumplen las restricciones temporales establecidas. Un setup de valor alto y positivo, indica que el diseño puede funcionar con una frecuencia de reloj mucho más alta.

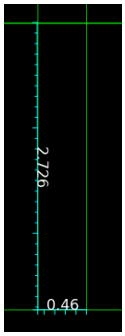
## 08-openroad-floorplan

TOP_LEVEL.pnl.v (Physical Netlist)

Misma cantidad de componentes, se agregó VPWR (alimentación positiva), VGND (tierra), VNB y VPB (conexiones al sustrato)



El área delimitada Core (area cuadrada) define el espacio total de silicio disponible para la lógica. Los rectángulos internos representan los sites o unidades mínimas de la grilla de construcción. Las celdas se ubican sobre estos sitios respetando su ancho específico, lo que garantiza que el diseño final sea compacto y cumpla con las reglas de fabricación de SkyWater 130nm.




| Architecture Comparison | Low Speed | High Density | High Density Low Leakage | High Voltage |
|---|---|---|---|---|
| TAP BAR | NO | NO | NO | YES |
| X-GRID | 0.480 | 0.460 | 0.460 | 0.480 |
| Y-GRID | 0.370 | 0.340 | 0.340 | 0.370 |
| CELL HEIGHT | 9 GRIDS | 8 GRIDS | 8 GRIDS | 11 GRIDS |
| CELL HEIGHT | 3.330 | 2.720 | 2.720 | 4.07 |
| NAND2 WIDTH | 3 GRIDS | 3 GRIDS | 4 GRIDS | 5 GRIDS |
| NAND2 WIDTH | 1.440 | 1.380 | 1.840 | 2.400 |
| NAND2 AREA | 4.7952 | 3.7536 | 5.0048 | 9.770 |
| WPMAX | 1.120 | 1.000 | 1.000 | 1.500 |
| WNMAX | 0.740 | 0.650 | 0.650 | 0.75 |

Se verificó mediante la herramienta de medición de OpenROAD que las dimensiones de los sites corresponden a la arquitectura High Density de la librería Sky130. La diferencia de CELL HEIGHT es por una mala ubicación del mouse.

```
"design__die__area": 2786.1,
"design__core__area": 1284.98,
"design__instance__count": 61,
"design__instance__area": 671.894,
"design__instance__count__stdcell": 61,
"design__instance__area__stdcell": 671.894,
```

Area total de silicio $= 2786{,}58\ \mu m^2$

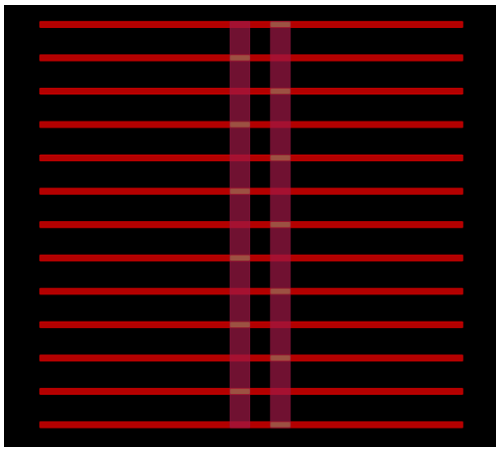
Area del Core $= 1284{,}98\ \mu m^2$

Area neta ocupada del Core $= 671{,}894\ \mu m^2$

$$\text{Utilización} = \frac{671{,}894\ \mu m^2}{1284{,}98\ \mu m^2}\ 100 = 52{,}2\ \%$$

## 14-openroad-generatepdn

No presenta errores en la generación de la red eléctrica

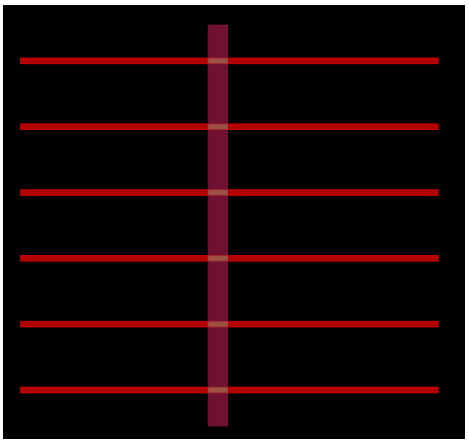


PDN (Power Distribution Network)

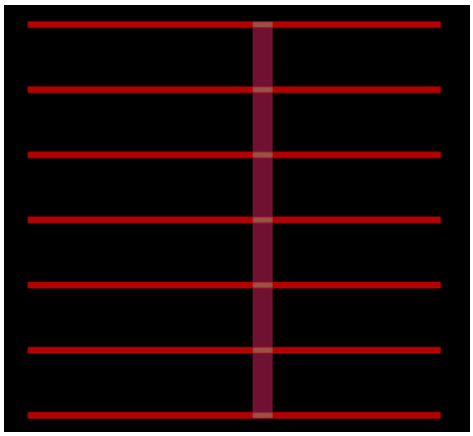
```
sky130_fd_sc_hd__nand2_2 _40_ (.A(_15_),
    .B(_16_),
    .VGND(VGND),
    .VNB(VGND),
    .VPB(VPWR),
    .VPWR(VPWR),
    .Y(_17_));
```

Los componentes ahora cuentan con una red de alimentación

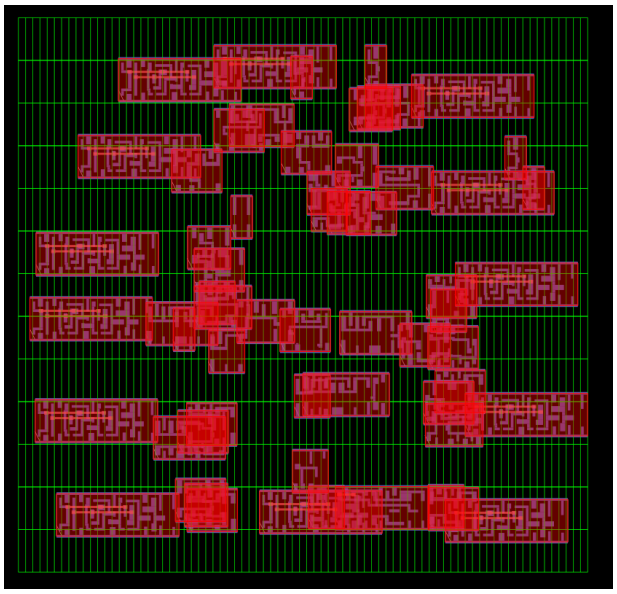
VPWR



VGND



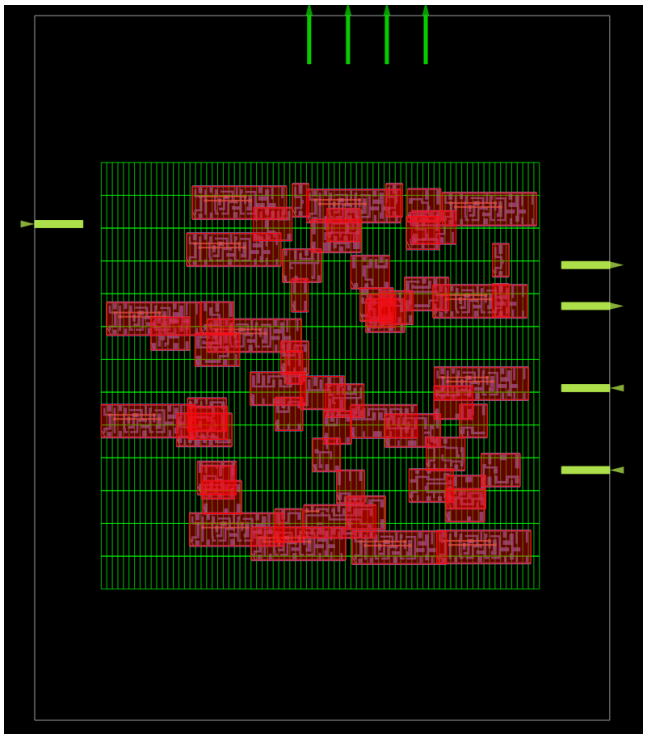
## 17-openroad-globalplacementskipio



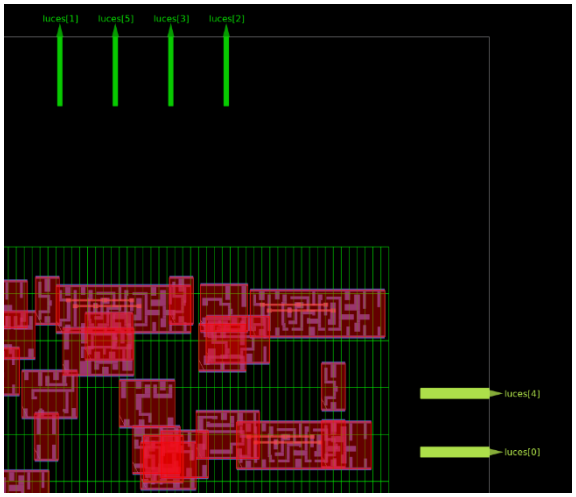
En esta etapa del flujo, se asigna una ubicación aproximada a las celdas dentro del core

```
- _047_ sky130_fd_sc_hd__or2_2 + PLACED ( 23008 15949 ) N ;
- _048_ sky130_fd_sc_hd__nor4b_2 + PLACED ( 23677 20848 ) N ;
- _049_ sky130_fd_sc_hd__nand2_2 + PLACED ( 24205 32633 ) N ;
- _050_ sky130_fd_sc_hd__a21o_2 + PLACED ( 22292 36275 ) N ;
```

## 18-openroad-ioplacement



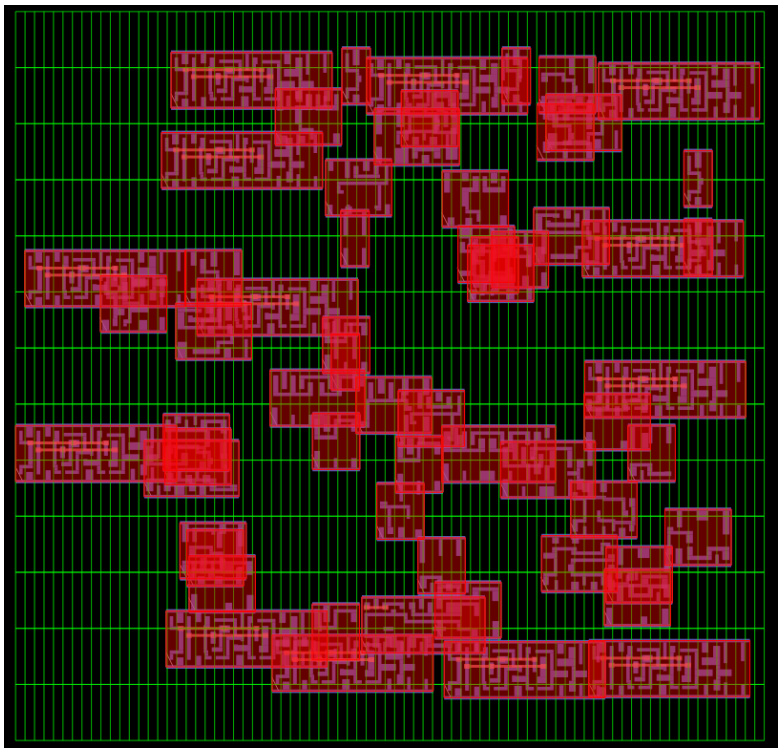
Se coloca los pines IO en el perímetro




## 21-openroad-globalplacement

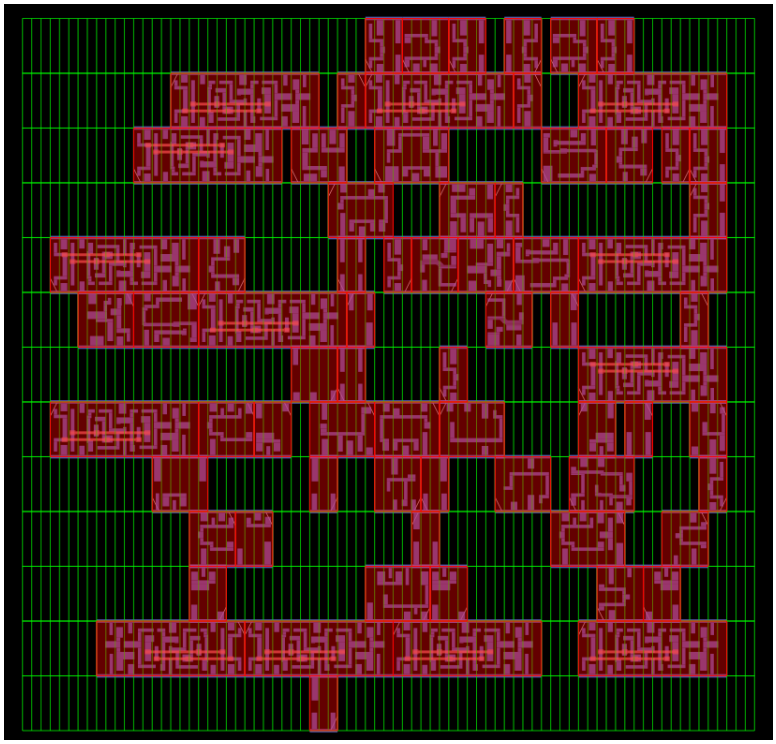
```
- _047_ sky130_fd_sc_hd__or2_2 + PLACED ( 23052 20637 ) N ;
- _048_ sky130_fd_sc_hd__nor4b_2 + PLACED ( 26206 23413 ) N ;
- _049_ sky130_fd_sc_hd__nand2_2 + PLACED ( 27634 32590 ) N ;
- _050_ sky130_fd_sc_hd__a21o_2 + PLACED ( 26225 35791 ) N ;
```

Se recoloca las celdas teniendo en cuenta la ubicación de los pines IO



26-openroad-detailedplacement

Se elimina el solapamiento, alineándose a los standards cell rows



27-openroad-cts y 28-openroad-stamidpnr-1

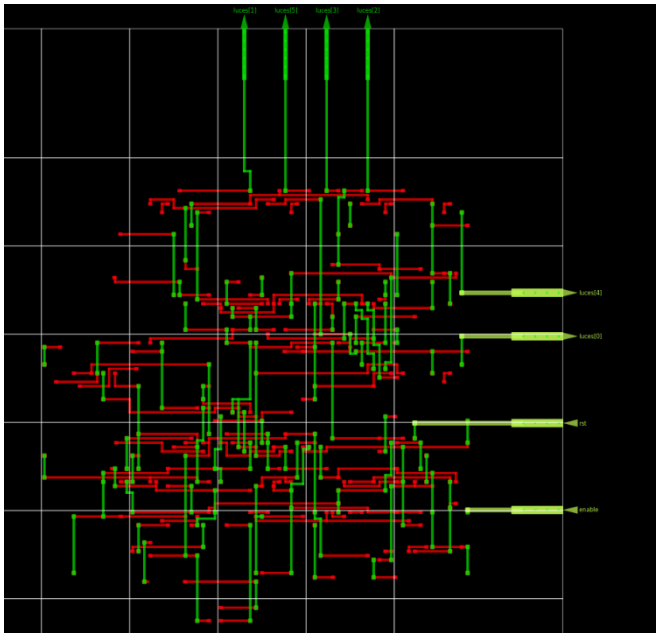
```
Clock: clk
Sources: clk
Generated: no
Virtual: yes
Propagated: no
Period: 20.000000
```

```
Clock clk
rise -> rise
    min      max
0.000000 0.000000 source latency
0.282306          network latency _101_/CLK
         0.282404 network latency _095_/CLK
---------------
0.282306 0.282404 latency
         0.000098 skew

fall -> fall
    min      max
0.000000 0.000000 source latency
0.287428          network latency _092_/CLK
         0.287491 network latency _096_/CLK
---------------
0.287428 0.287491 latency
         0.000063 skew
```

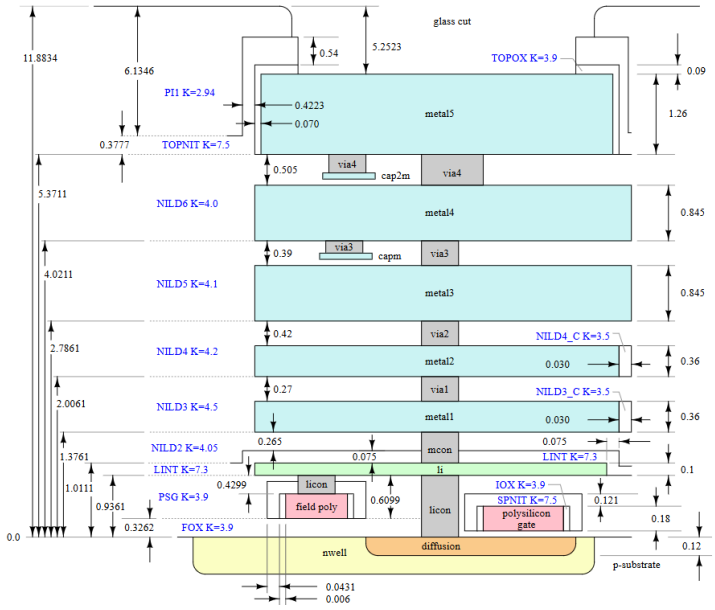
```
=========================================
report_clock_min_period
=========================================
clk period_min = 1.85 fmax = 539.33
```

El paso 27-openroad-cts genera el árbol de reloj (Clock Tree Synthesis, CTS), es decir, la red física del clock. El paso 28-openroad-stamidpnr-1 se genera un reporte de temporización en el que el flujo aún considera valores ideales para el clock. Allí se realizan los cálculos de latencia y se observa que la diferencia de tiempo de llegada del clock entre un flip-flop y otro es de aproximadamente 98 ps, lo que indica un skew muy bajo. Además, el diseño admite un período mínimo de 1,85 ns, lo que permite alcanzar una frecuencia máxima cercana a 539 MHz.

## 31-openroad-globalrouting y 36-openroad-detailedrouting



En el paso 31 se realizó la planificación global del enrutamiento, en el paso 36 se ejecutó el enrutamiento detallado, generando las pistas metálicas visibles en el layout. Se observa en color verde a metal 2, mientras que lo que aparece en color rojo corresponde a metal 1.




## 47-openroad-stapostpnr

Se observa que no existen problemas de *slack*. Los valores de *Worst Slack* son positivos y no se registran violaciones de *setup* ni de *hold* en ninguno de los *corners* analizados. Esto indica que el diseño cumple las restricciones temporales para las distintas condiciones de tensión y temperatura evaluadas.

| Corner/Group | Hold Worst Slack | Reg to Reg Paths | Hold TNS | Hold Vio Count | of which reg to reg | Setup Worst Slack | Reg to Reg Paths | Setup TNS | Setup Vio Count | of which reg to reg | Max Cap Violatio… | Max Slew Violati… |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Overall | 0.1198 | 0.1198 | 0.0000 | 0 | 0 | 12.9863 | N/A | 0.0000 | 0 | 0 | 0 | 3 |
| nom_tt_025C_1v80 | 0.3434 | 0.3434 | 0.0000 | 0 | 0 | 14.5895 | N/A | 0.0000 | 0 | 0 | 0 | 0 |
| nom_ss_100C_1v60 | 0.9439 | 0.9439 | 0.0000 | 0 | 0 | 12.9936 | N/A | 0.0000 | 0 | 0 | 0 | 3 |
| nom_ff_n40C_1v95 | 0.1219 | 0.1219 | 0.0000 | 0 | 0 | 15.0232 | N/A | 0.0000 | 0 | 0 | 0 | 0 |
| min_tt_025C_1v80 | 0.3404 | 0.3404 | 0.0000 | 0 | 0 | 14.5971 | N/A | 0.0000 | 0 | 0 | 0 | 0 |
| min_ss_100C_1v60 | 0.9387 | 0.9387 | 0.0000 | 0 | 0 | 13.0044 | N/A | 0.0000 | 0 | 0 | 0 | 3 |
| min_ff_n40C_1v95 | 0.1198 | 0.1198 | 0.0000 | 0 | 0 | 15.0266 | N/A | 0.0000 | 0 | 0 | 0 | 0 |
| max_tt_025C_1v80 | 0.3460 | 0.3460 | 0.0000 | 0 | 0 | 14.5858 | N/A | 0.0000 | 0 | 0 | 0 | 0 |
| max_ss_100C_1v60 | 0.9487 | 0.9487 | 0.0000 | 0 | 0 | 12.9863 | N/A | 0.0000 | 0 | 0 | 0 | 3 |
| max_ff_n40C_1v95 | 0.1238 | 0.1238 | 0.0000 | 0 | 0 | 15.0191 | N/A | 0.0000 | 0 | 0 | 0 | 0 |

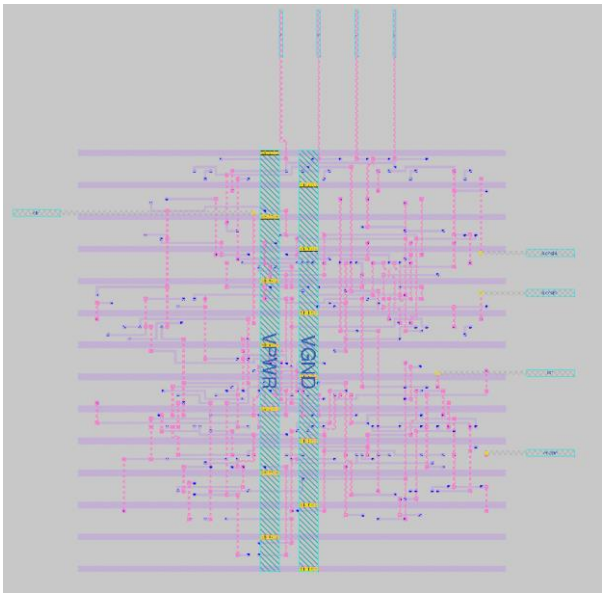
48-openroad-irdropreport

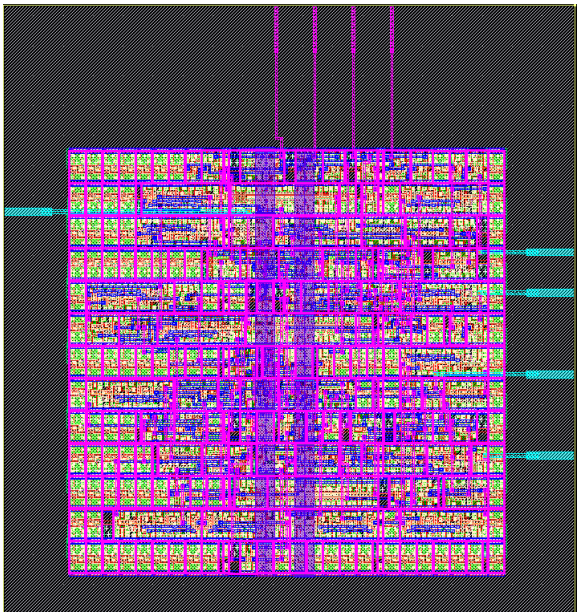
```
[INFO PSM-0040] All shapes on net VPWR are connected.
########## IR report #################
Net                 : VPWR
Corner              : nom_tt_025C_1v80
Supply voltage      : 1.80e+00 V
Worstcase voltage: 1.80e+00 V
Average voltage     : 1.80e+00 V
Average IR drop     : 1.62e-05 V
Worstcase IR drop: 3.17e-05 V
Percentage drop     : 0.00 %
######################################
[INFO PSM-0040] All shapes on net VGND are connected.
########## IR report #################
Net                 : VGND
Corner              : nom_tt_025C_1v80
Supply voltage      : 0.00e+00 V
Worstcase voltage: 3.56e-05 V
Average voltage     : 1.44e-05 V
Average IR drop     : 1.44e-05 V
Worstcase IR drop: 3.56e-05 V
Percentage drop     : 0.00 %
```

Se verificó que todas las piezas de metal (shapes) que forman la red de alimentación están físicamente conectadas bajo las condiciones de temperatura y voltaje bajo en las cuales se realizó el análisis. Donde Average IR drop indica la caída media en toda la red, $16.2\ \mu V$ , la cual es insignificante mientras que Worstcase IR drop corresponde a la caída máxima en algún punto específico de la red, $31.7\,\mu V$, prácticamente despreciable.

49-magic-streamout y 50-klayout-streamout

Se obtuvo el layout final con Magic y KLayout. El diseño ya está fijo, los pasos posteriores del flujo son de comprobación y verificación, pero físicamente el diseño ya no cambiará



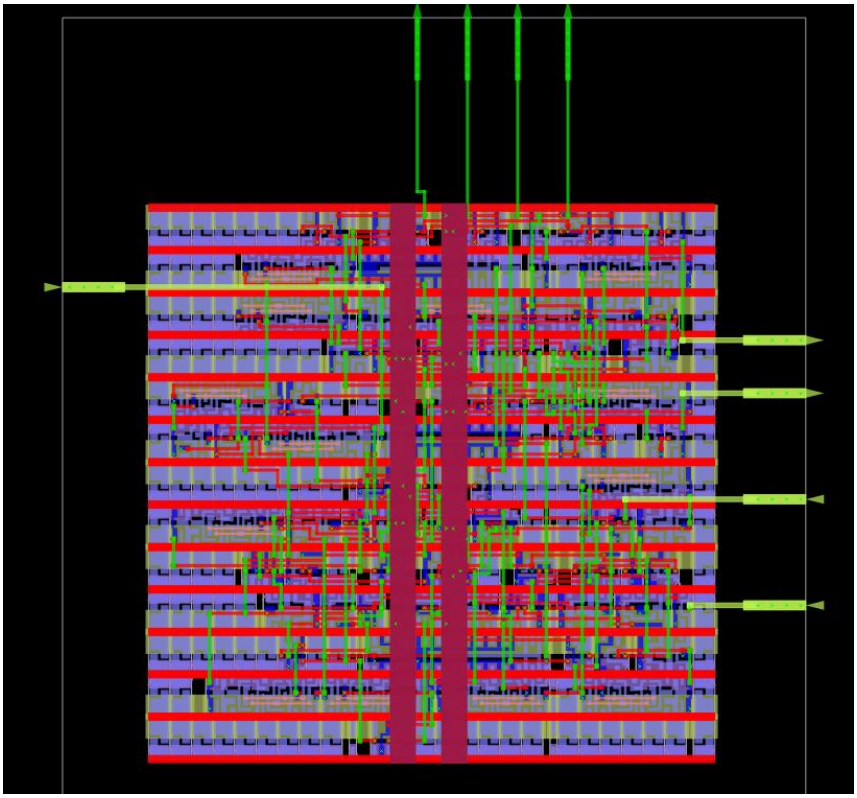



67-misc-reportmanufacturability

```
* Antenna
Passed ✅

* LVS
Passed ✅

* DRC
Passed ✅
```

La verificación de manufacturabilidad del diseño fue completada con éxito, los chequeos de DRC, LVS y Antenna se realizaron sin errores, confirmando que el layout final cumple con las reglas de diseño y coincide con la netlist

final



## *Conclusión*

La materia Análisis y Diseño de Circuitos Integrados constituye una formación integral que abre múltiples posibilidades laborales en distintos ámbitos de la ingeniería, dado que actualmente los circuitos integrados y ASICs se encuentran presentes en una amplia variedad de aplicaciones, como electrónica, biomédica, telecomunicaciones, la industria automotriz, los sistemas embebidos y los dispositivos de consumo masivo, entre otros. El diseño de chips es hoy un área estratégica dentro del desarrollo tecnológico, y un profesional capacitado en este campo cuenta con importantes oportunidades de crecimiento. Si bien en la actualidad existen herramientas automatizadas que permiten ejecutar gran parte del flujo de diseño, esto no significa que el proceso sea simple. Incluso en diseños relativamente pequeños, surgen desafíos constantes, ya que no solo debe garantizarse la funcionalidad lógica del circuito, sino también el cumplimiento de restricciones físicas, eléctricas y temporales para asegurar su correcto funcionamiento en silicio. En el presente proyecto, aunque no se exigieron especificaciones particularmente complejas, el desarrollo representó un desafío significativo. Se pudo comprobar que un detalle aparentemente menor como un valor incorrecto o la omisión de una señal de reloj puede alterar de manera considerable el resultado final del diseño. Por ello, el aprendizaje teórico y la práctica en esta materia resultan fundamentales. Si en un proyecto de nivel introductorio pueden presentarse múltiples dificultades, en un diseño de mayor escala el riesgo de errores aumenta considerablemente. En consecuencia, es imprescindible prestar atención a cada detalle y realizar verificaciones exhaustivas en todas las etapas del flujo de diseño.

# *Referencias*


- LibreLane Documentation. (s.f.). *LibreLane Documentation*. Read the Docs. https://librelane.readthedocs.io/en/latest/index.html

- OpenLane 2 Documentation. (s.f.). *OpenLane 2 Documentation*. Read the Docs. https://openlane2.readthedocs.io/en/latest/index.html

- SkyWater SKY130 PDK Documentation. (s.f.). *SkyWater SKY130 PDK Documentation*. Read the Docs. https://skywater-pdk.readthedocs.io/en/main/ (skywater-pdk.readthedocs.io

- Docker, Inc. (s.f.). *Docker Desktop: The #1 containerization tool for developers*. Docker. https://www.docker.com/products/docker-desktop/

- The OpenROAD Project. (s.f.). *The OpenROAD Project – Foundations and Realization of Open and Accessible Design*. https://theopenroadproject.org/

- Weste, N. H. E., & Harris, D. M. (2011). *CMOS VLSI design: A circuits and systems perspective* (4th ed.). Addison-Wesley.

- Maxinez, D. G., & Alcalá Jara, J. (2002). *VHDL: El arte de programar sistemas digitales* (1.ª ed.). Instituto Tecnológico y de Estudios Superiores de Monterrey, Campus Estado de México